

CS 313 - Project 3

Binary Search Tree

Deadline: 11:59PM, Wednesday, November 19, 2025

1 Project Overview

For this project, we will be taking what we've learned in class and using it to implement a Binary Search Tree (BST). In CS, a binary search tree is used in a variety of application domains from routers to data compression. We will focus on building the simple BST to prepare us for a self-balancing binary tree such as a red-black tree.

2 Program Requirements:

2.1 Write `BinarySearchTree` class.

For convenience, a `TreeNode` class is already provided in the starter code. Your job is to implement the following `BinarySearchTree` class. To receive any credit, your class must follow the specifications below exactly:

- Class Name: **`BinarySearchTree`**
- Methods:
 - `__init__(< BinarySearchTree > self) → < Nonetype > None`
 - * **[5 points]**
 - * Complexity: $O(1)$.
 - * Instance variables:
 - `< TreeNode > root`: This variable contains the root of our tree. Note: the tree is initially empty.
 - `insert(< BinarySearchTree > self, < int > value) → < Nonetype > None`
 - * **[30 points]**
 - * Complexity: $O(\lg(n)) \sim O(n)$.
 - * Valid Input: An integer in the bound $(-\infty, +\infty)$.
 - * Description: The `insert` method will insert a new item into the tree. It will do so by creating a new node with the given integer then adding it to the tree. This should also preserve the ordering requirement that all nodes on the right side of a subtree are greater than the value in the root of that subtree, and all elements on the left side are lesser than the value in the root of the subtree.
 - * Note: To simplify things, we will not test your binary search tree with duplicate elements.
 - `delete(< BinarySearchTree > self, < int > value) → < TreeNode > returnValue`
 - * **[30 points]**
 - * Complexity: $O(\lg(n)) \sim O(n)$
 - * Valid Input: An integer in the bound $(-\infty, +\infty)$.

- * Description: The *delete* method will remove a node from the tree with the given value and return it.
- *find*($\langle BinarySearchTree \rangle self, \langle int \rangle value$) $\rightarrow \langle bool \rangle returnValue$
 - * **[20 points]**
 - * Complexity: $O(\lg(n)) \sim O(n)$
 - * Valid Input: An integer in the bound $(-\infty, +\infty)$.
 - * Description: This method takes an integer *value* and returns True if there exists a node in the tree that matches that value. If no such node exists in the tree, return False.
- *inorder_traversal*($\langle BinarySearchTree \rangle self$) $\rightarrow \langle list \rangle returnValue$
 - * **[15 points]**
 - * Complexity: $O(n)$
 - * Description: This method performs the inorder traversal for the binary search tree. It should return a Python list with the values of all the nodes in the tree in the increasing order.

3 Submission Requirements:

Submit a single file **lab3.py** to canvas.